Standard Quant Tools

Quantitative finance for AI agents

Measured, Not Projected

Typed tools · One runtime per agent · Point-in-time by declaration · Every call on the record

---

TECHNICAL BRIEFING

What Standard Quant Tools is, how it is built, what it computes, and the evidence behind it

A briefing for engineers, quants and anyone wiring an agent to the library. Every figure in it was measured from the code on 27 October 2026, at commit `09727c7`. Where something was measured and did not help, the document says so.



Karan Vora, Maintainer · Apache 2.0

github.com/karanvora2599/Standard-Tools

Contents

—	Executive summary	3
I	**PART I · WHY IT EXISTS**	5
01	The problem: numbers nobody computed	6
02	The approach: the agent chooses, the library computes	7
II	**PART II · ARCHITECTURE**	8
03	System overview	9
04	Twelve runtimes, enforced at dispatch	10
05	Typed references between runtimes	11
06	The MCP server and the context budget	12
III	**PART III · WHAT IT COMPUTES**	13
07	Data, and when a fact became knowable	14
08	Research and statistical inference	15
09	Modeling	16
10	Backtesting, and how much of it is real	17
11	Portfolio, allocation and finance	17
12	Derivatives, delta one and microstructure	18
IV	**PART IV · EVIDENCE**	19
13	The audit trail	20
14	The correctness audits	20
15	The testing regime	21
16	Performance, measured	22
17	Where it did not help	23
V	**PART V · USING IT**	24
18	Getting started	25
19	Limits, stated plainly	26
APP.	**APPENDICES**	27
A	Appendix A · Numbers at a glance	28
B	Appendix B · Glossary	29
C	Appendix C · The complete tool catalog	29

Executive summary

Standard Quant Tools is a Python library for quantitative finance, built to be called by AI agents. Every tool takes one Pydantic model and returns one, so an agent gets structured data back, and errors that say what to fix. The CPU-bound paths run in an optional C++ extension behind the same API, with a pure-Python fallback when it is not built.

THE PROBLEM

A language model produces the most plausible next number, not a computed one. A Sharpe ratio, a hedge ratio or a volatility figure written by a model reads exactly like one that was calculated, and in finance a wrong number is a position. A model that is handed a library meant for people also gets untyped returns, silent defaults and hundreds of functions it was never meant to call.

THE APPROACH

The agent chooses; the library computes. **266 tools** sit in **twelve runtimes**, each a separate dispatch table, and a client is served one runtime rather than the whole surface. Inputs are validated, unknown arguments refused, bulk values passed between runtimes as typed `sqt://` references, and a call through `dispatch()` can leave a hash-chained decision record that replays it.

WHAT MAKES IT DIFFERENT

- **Scope is enforced, not suggested.** A tool outside a runtime is unroutable, and the refusal names the runtime that owns it.
- **Point-in-time by declaration.** A source says when its facts became knowable, and a join that would use hindsight is refused rather than quietly allowed.
- **Tested against planted answers.** Nine test layers, from known-answer correctness to mutation testing, each catching a class of defect the others cannot.
- **Negative results are kept.** Ports that did not pay off, and rules that did not help on an honest test, are reported beside the ones that did.

266
LLM-callable tools, each Pydantic in and Pydantic out, with function-calling schemas

12
runtimes, each an execution boundary that refuses the tools it does not own

19,256
Python tests in nine layers, plus 16 C++ test executables with 95,816 checks

274×
rolling Hurst with the C++ extension, against the same module's Python path

727 KB
of schema on the whole surface, about 186,000 tokens: the reason a client is served one runtime

21 / 21
planted mutations killed: the suite notices a defect, not only that it runs

STATUS, STATED PLAINLY

Version 0.1.0 under Apache 2.0, 595 commits since 15 February 2026, CI green on Python 3.10, 3.11 and 3.12. The C++ extension builds wherever a compiler is present and is optional everywhere: without it every tool runs its Python path and gives the same answer. Live vendor behaviour is established only with a live key, and the suite says which tests need one.

WHAT THIS DOCUMENT IS NOT

It is not investment advice and not a performance claim. A test can show a statistic is computed correctly; it cannot show that a strategy makes money. Where a figure comes from planted data, it says so.

Three things to remember

1 · THE MODEL IS NOT TRUSTED TO COMPUTE

It chooses a tool and reads its result. Every number comes from a typed, deterministic function whose code is open.

2 · EVERY CALL CAN BE RECONSTRUCTED

A decision record holds the inputs, the content hashes of the data read and a hash of the output, so a call can be replayed and compared.

3 · IT SAYS WHAT IT CANNOT DO

A refusal names what was wrong and what to change. An in-sample number is labelled in-sample, and an untested option is opt-in.

HOW TO READ THIS BRIEFING

Part I is the problem and the approach. **Part II** is the architecture: the five stages of a call, the twelve runtimes, the references between them and the MCP server. **Part III** is what the library computes, area by area. **Part IV** is the evidence: the audit trail, the audits, the tests, the measured speed and where things did not help. **Part V** is how to start and what the library does not do. **Appendix C** lists every tool, generated from the registry.



PART I

Why it exists

An agent can describe a portfolio fluently and still never have computed a single number in it. This library exists so that every number it reports came from a function that can be read, tested and replayed.

01 The problem: numbers nobody computed

02 The approach: the agent chooses, the library computes

01 The problem: numbers nobody computed

A language model produces the most plausible continuation of its context. A volatility, a Sharpe ratio or a basis-point spread is just another plausible continuation, and nothing in the text separates a figure that was *computed* from one that was *written because it looked right*.

Handing the model a quant library does not fix this by itself. Libraries written for people return untyped frames, fill gaps with silent defaults and expose every function at once. A model given all of it picks tools it should never have seen, passes arguments that are quietly ignored, and joins a quarterly filing on the date it describes rather than the date it was published. Each mistake yields a number that looks fine.

Six ways a computed answer still goes wrong

FAILURE	WHAT IT LOOKS LIKE	WHAT THE LIBRARY DOES ABOUT IT
Fabricated figure	"30-day realized vol is 24.1%", with no tool run behind it.	Every number is a tool's output, a Pydantic model the agent reads and cites.
Look-ahead by join	A filing describes 30 September and is published on 25 October. Joined on the quarter end, every row carries three weeks of hindsight.	Each source declares a temporal contract; a join it cannot support is refused before anything is fetched.
Scope creep	A research agent runs a backtest, or a sizing tool it was never given.	Twelve runtimes, each its own dispatch table. A tool outside the runtime is unroutable.
Ignored argument	A misspelled parameter falls back to a default and the result looks normal.	Unknown arguments are rejected, and the error names what was wrong and what to change.
Unreplayable work	Last week's figure cannot be reproduced, because the data or the code moved.	A decision record keeps the inputs, the data's content hashes and the output hash; `sqt replay` says whether it still reproduces.
Overfitted result	The best of forty backtests is reported as if it were the only one.	Deflated Sharpe, PBO, purged cross-validation and the reality check, each taking the trial count.

None of these is exotic. Each is ordinary code or an ordinary request that looks right, and the cost of each is a confident number with nothing behind it. The rest of this document is how the library makes each one either impossible or visible.

02 The approach: the agent chooses, the library computes

The split of work is strict. The model decides which question to ask and which tool answers it; the library does every piece of arithmetic, with typed inputs, typed outputs, and a record of what it read.

TYPED AT BOTH ENDS

Each tool is one input model and one result model, both JSON-serializable for function calling. An input the model cannot express is an input the agent cannot get wrong.

SCOPED BY RUNTIME

A client is served one of twelve runtimes, not the whole surface. Scope is enforced at dispatch, so a tool an agent was never given is unroutable rather than merely unlikely.

HONEST BY DEFAULT

Every Sharpe-reporting tool takes the risk-free rate it is measured against. In-sample figures are labelled in-sample. Options that did not help on an honest test stay opt-in.

RECORDED AND REPLAYABLE

A call through `dispatch()` can write a decision record, hash-chained across calendar days, that `sqt replay` re-runs and `sqt verify` checks.

Errors are part of the interface

A refusal is written for the model that will read it. It says what was wrong, which value would be accepted, and, for a tool from another runtime, which runtime owns it. The fuzzing layer of the tests (section 15) holds every tool to that contract for *any* input: a named refusal or a valid result, never a stack trace.

THE SAME CODE FOR PEOPLE

Nothing here is agent-only. The same functions are importable from Python: indicators, metrics, backtests, optimizers and models, with the same validation. The agent surface is a typed layer over the library, not a separate implementation of it.



PART II

Architecture

Five things happen to every call, in order. Twelve runtimes decide which calls are possible at all, typed references carry results between them, and an MCP server serves one runtime at a time.

03 System overview

04 Twelve runtimes, enforced at dispatch

05 Typed references between runtimes

06 The MCP server and the context budget

03 System overview

One package, `standard_quant_tools`, importable from Python and served over MCP. Under its agent surface sit the same indicators, metrics, backtests, optimizers and models a person would call directly, and under the CPU-bound ones an optional C++ extension.

The five stages of a call

SERVE	A client is given one runtime as MCP tools or function-calling schemas · under `--tool-detail auto` the heaviest schemas are thinned and `describe_tool` returns any of them in full
DISPATCH	The input is validated against the tool's Pydantic model · unknown arguments are refused · a tool from another runtime is unroutable, and the refusal names its owner
COMPUTE	C++ kernels when the extension is built, the Python path when not, behind one API · sources declare when their facts became knowable · Sharpe is measured against a stated risk-free rate
HAND OFF	Equity curves, trade logs, return panels and predictions are published as `sqt://` references · their kind is checked when another runtime resolves them · nothing large passes through the conversation
RECORD	A decision record per dispatched call: inputs, the content hashes of the data read, the execution path and a hash of the output, hash-chained across calendar days · `sqt replay` and `sqt verify`

From Python, a tool is one function call with a Pydantic model in and out. Over MCP, `sqt-mcp --runtime research` serves one runtime to any client.

Scale of the build

~198k
lines of Python in the package, in 379 files

~14.6k
lines of C++ in the optional extension, in 40 files

~193k
lines of tests in 530 Python files, plus 18 C++ test sources

595
commits since the first, on 15 February 2026

31
modeling features, with 27 estimators (8 of them optional) and 18 target kinds

15
tool categories, which narrow within a runtime rather than across them

04 Twelve runtimes, enforced at dispatch

A runtime is an execution boundary, not a label. Each holds its own dispatch table, so a tool from another runtime cannot be called through it at all, and the refusal names the runtime that does own it.

That makes a scoping mistake recoverable and a hallucinated tool name obviously different from one. An agent serving research questions cannot run a backtest by accident, because the backtest runtime's tools are not in its table. A category narrows *within* a runtime (`--categories microstructure` is not the same as `--runtime microstructure`), and fifteen categories sit across the twelve.

RUNTIME	TOOLS	SCHEMA	WHAT IT ANSWERS
research	46	75 KB	Describe an asset or a universe: screen it, profile its risk and technicals, and analyze its statistical structure (factors, cointegration, PCA,...
modeling	38	213 KB	Build, validate and score a statistical model from this library's own features: dataset construction, leakage-purged walk-forward fitting, the model...
backtest	35	102 KB	Run, optimize, validate and diagnose a trading strategy — the library's built-in ones or a signal the caller computed themselves.
meta	25	25 KB	Questions about the library and the session rather than about a market: what this library accepts and what the data provider can serve, and what a...
portfolio	23	57 KB	Turn a view into a position and price what it costs.
data	21	36 KB	Get the data and publish it as an `sqt://` reference every other runtime can read: OHLCV for one name or a whole universe, return panels, tick tapes...
delta_one	18	46 KB	Which instrument is the cheapest way to own or hedge this exposure.
microstructure	17	31 KB	What the market will charge you to trade, at two data fidelities.
derivatives	13	26 KB	Price an option and understand what holding it does to you: the second-order greeks, multi-leg payoffs, the consistency of a quoted surface, what...
feature_lab	11	50 KB	Interrogate the FEATURES of a built dataset, before and independently of fitting anything: what each one measures and predicts, which are...
allocation	10	43 KB	How much capital each strategy, sleeve or model deserves, from their return streams rather than from tickers: the streams on one calendar, an...
finance	9	23 KB	What holding a book costs and what a stream of cash is worth: financing, borrow and cash drag on positions, the Reg T and FINRA margin against them...
all	**266**	727 KB	The whole surface, which no client is served at once.

Schema is the bare bytes of each runtime's tool schemas at full detail, as `sqt-mcp --print-budget` reports them. 217 of the 266 tools also sit in the analysis facade, `get_agent_tools()`.

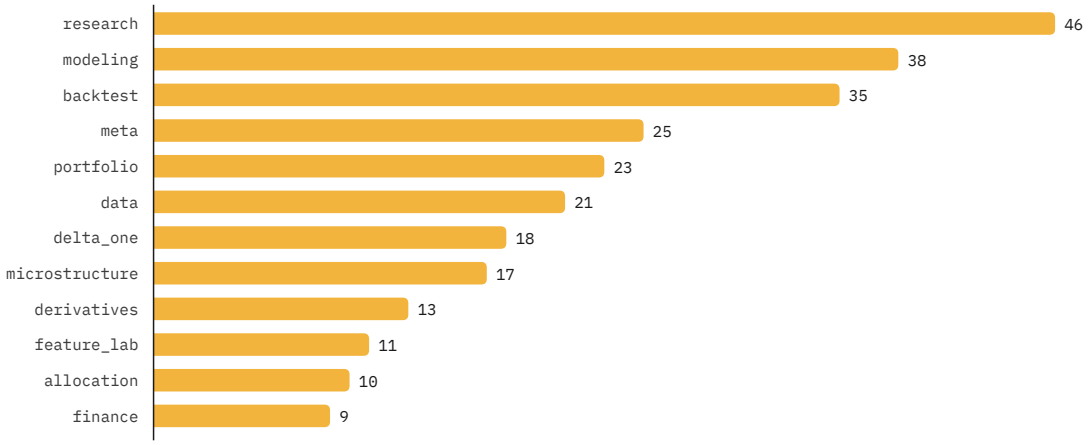


Figure 1. Tools per runtime, read from the live registry on 27 October 2026. Appendix C lists every one.

Why a boundary and not a filter

Narrowing the schemas a model is shown does not, by itself, stop a dispatcher from executing whatever name it is handed: a model shown two research tools that names a backtest would still get a backtest. Here the narrowing and the execution are the same table: scope is enforced where the call runs, so what an agent was not given it cannot do. Registering a tool touches six places (the function, its input and result models, the runtime's definitions and dispatch, and the facade), and a whole-surface test (section 15) fails when one is missed.

05 Typed references between runtimes

Runtimes isolate execution, not data. Every real workflow spans them: screen in research, backtest in backtest, size in portfolio, hand a model's predictions to an allocation. Bulk values cross as typed references.

```
sqt://<kind>/<run_id>/<name>
```

A producer publishes a bulk value under a `run_id` and gets back a string. Any consumer in any runtime resolves it. Neither side knows the other exists, so N producers and M consumers need N + M integrations instead of N × M bespoke bridges, and nothing is transcribed through a context window on the way.

```
handoff.resolve(ref, expect="equity_curve")
# ValidationError: expected an 'equity_curve' reference but
# 'sqt://trade_log/run1/trades' is a 'trade_log'. ...
```

THE KIND IS CHECKED ON RESOLVE

A consumer states the kind it expects. Handing a trade log to a tool that wants an equity curve fails with an error naming both kinds, instead of producing a drawdown computed from the wrong series.

REFERENCES ARE TOOLS TOO

`describe_reference`, `read_reference`, `convert_reference` and `list_reference_kinds` live in the meta runtime, so any agent can inspect what it was handed.

06 The MCP server and the context budget

The 266 tools cost about 727 KB of schema, roughly 186,000 tokens, held for a whole session. That is the constraint the server is designed around, so what to serve is the first decision rather than a tuning knob.

At full detail a tool averages 3,132 bytes over the wire. A session ceiling of 180,000 bytes, which the server once had, buys about 57 tools; taken smallest first, only 127 of the 266 fit. No amount of schema-shrinking brings the whole surface back, which is why a client is served one runtime, and why `--runtime all` is a diagnostic rather than a deployment. There is no fixed ceiling now: what a client can afford is a property of the client.

THINNING, WITH THE FULL SCHEMA ONE CALL AWAY

`--tool-detail auto` is the default. It thins the most expensive schemas until a runtime fits, and injects `describe_tool`, which returns any thinned schema in full. Served sizes under `auto`:

RUNTIME	TOOLS	SERVED
`modeling`	38	43 KB
`research`	46	46 KB
`backtest`	35	43 KB
`microstructure`	17	36 KB
`allocation`	10	31 KB
`derivatives`	13	30 KB

WHAT A SESSION HOLDS

- **The default session** serves four categories (screening, analysis, quantitative research and discovery): 62 tools, about 23,000 tokens of schema.
- **Resources** expose `sqt://` references, and **prompts** package common workflows.
- **Long-running tools**, `run_backtest_optimization` and `scan_pairs`, are served only with `--enable-long-running`.
- **The audit trail** covers calls made over the protocol, through the same dispatch as calls made from Python.

Served bytes are the envelope the server sends; the table in section 04 is bare schema bytes. The two are different instruments and are not comparable line for line.



PART III

What it computes

From getting the bytes to splitting capital across strategies: each area is a runtime with its own question, and each states what its numbers can and cannot support.

07 Data, and when a fact became knowable

08 Research and statistical inference

09 Modeling

10 Backtesting, and how much of it is real

11 Portfolio, allocation and finance

12 Derivatives, delta one and microstructure

07 Data, and when a fact became knowable

Fetching is a runtime of its own, 21 tools that get the bytes and publish them as references every other runtime reads. Before anything is fetched, a source can be asked what it can promise about *when* its facts became knowable.

PROVIDERS

A provider-agnostic interface serves OHLCV, fundamentals and metadata, with a free default and adapters for Polygon, Bloomberg and Databento. Databento is the one here that serves L2 depth and order-by-order events, and its key is read from the vendor's own variable, never passed through a spec, so it cannot be persisted into a model's lineage or a decision record. `preflight_vendor_request` says what a metered request would cost, and whether the data is there, before it is made.

INFRASTRUCTURE

Retries with exponential backoff that never retry a caller's own mistake, a TTL and Parquet cache, and quality checks for missing bars, stale prices and price jumps. `get_dataset_metadata` reports what the provider guarantees: adjustment, survivorship, point-in-time values and the timezone.

THE TEMPORAL CONTRACT

A quarterly filing describes 30 September and is published on 25 October. Joined on the quarter end it puts three weeks of hindsight into every row. `describe_temporal_contract` answers what a source can say about timing before anything is fetched, and a join the contract cannot support is refused rather than degraded silently. `infer_temporal_contract` does the same for a frame this library did not fetch.

BUNDLES AND COMPARISONS

A bundle holds several frames and the contract each was fetched under, validated as a unit. `compare_ratio_frames` classifies two providers' disagreement: a unit mismatch can be rescaled, a definition difference cannot.

DATA TOO LARGE TO FETCH

`register_external_dataset` makes a Parquet or CSV dataset already on disk resolvable as an `sqt://` reference without copying it, and `prepare_vendor_extract` converts a raw vendor extract into the library's contract first. Continuous futures series, tick tapes, quote panels and order-book snapshots are published the same way.

08 Research and statistical inference

The research runtime, 46 tools, describes an asset or a universe: its risk and technicals, and its statistical structure. It does not run strategies. Around it sits a family of error bars and tests, so a summary statistic arrives with how far it can be trusted.

AREA	TOOLS
Structure	Factor regressions, cointegration and pair scans, PCA, Hurst, regimes, change points, structural breaks, a Kalman hedge ratio, GARCH volatility forecasts
Error bars	Block bootstrap intervals, Sharpe stability, drawdown profiles, distribution comparison, normality, tail index and tail dependence
Dependence	Granger causality, autocorrelation, stationarity, lead-lag matrices, partial correlation, correlation stability, seasonality, entropy
Risk	Tail and extended risk metrics, volatility estimators beside close-to-close, stock risk against a benchmark, data quality reports

A factor model, and which factors belong in it

`estimate_factor_model` fits loadings, a factor covariance and specific variances for many assets at once, with standard errors, and can publish B F B' + D as a covariance reference. `screen_factor_candidates` decides which candidate factors earn a place, judged out of sample: incremental R-squared against circularly shifted copies (Benjamini-Hochberg adjusted across candidates), stable loadings, a low VIF, and a minimum-variance book that is less volatile with the factor than without.

0 of 4,000
noise candidates selected on planted data

39–62%
how often a weak real factor was selected

31–34 bp
a year less volatile than Ledoit-Wolf, for the selected model's minimum-variance book

VaR as a forecast that can be checked

`forecast_var_series` makes a one-step-ahead VaR and expected shortfall for every date from the returns before it, by six methods: historical, normal, Cornish-Fisher, EWMA, filtered historical, and EVT on GARCH. `backtest_var_forecasts` tests the series against what followed: Kupiec, Christoffersen, conditional coverage, the Basel traffic light, and Acerbi-Szekely for ES. A model's own quantile column can be tested the same way, entity by entity, with overlapping multi-day outcomes thinned.

09 Modeling

A separate 38-tool runtime builds, validates and scores statistical models as one ordered pipeline: dataset, leakage checks, walk-forward fitting, the registry, and evaluation of out-of-sample predictions. A feature lab of 11 more tools interrogates features before anything is fitted.

WHAT IT IS BUILT FROM

31 features, 27 estimators (19 always available and 8 that register only when their library is installed), 10 preprocessing steps and 18 target kinds, six of them buildable from prices. A feature matrix computed elsewhere can be registered without copying it.

LEAKAGE, PURGED BY DATE

Training rows are purged by each row's label end *date*, not by an integer offset: a horizon counts an entity's own bars, and on a sparse calendar an offset under-purges exactly where it matters. Point-in-time joins, `check_leakage` and a gate on each model's training information cutoff close the rest.

A REGISTRY WITH A LIFECYCLE

Models are content-addressed, with signed manifests: `attest_model_package` says whether a package is still the one registered and whether a trusted key signed it. Promotion through lifecycle stages, drift monitoring and a bridge from predictions to a tradeable signal complete it.

PREDICTIONS INTO PORTFOLIOS

`evaluate_model_portfolio` turns out-of-sample predictions into a simulated book net of costs. The `mean_variance` transform sizes it with the optimizer, trading from the book actually held after prices drift.

WHAT THE OPTIMIZER-SIZED TRANSFORM WAS MEASURED TO DO

On planted data, net of costs: with a real edge it ties rank weighting damped to half turnover (+0.01 ± 0.05 Sharpe). Where correlated names score alike it wins (+0.14 ± 0.07). With no edge it loses 0.26% a year, against 2.9% for damped rank. Trading from the drifted book added 0.02–0.03 Sharpe and cut no-edge turnover by a third.

10 Backtesting, and how much of it is real

The backtest runtime, 35 tools, runs, optimizes and validates strategies, the built-in ones or a signal the caller computed. Eleven of them exist to answer a question a backtest cannot answer about itself: how much of the result is real.

RUNNING

A vectorized engine with realistic costs, built-in strategies and parameter grids, a signal-panel backtester that accepts the caller's own matrix, pairs and futures backtests, walk-forward runs, Monte Carlo, and a shared-cash portfolio simulator with pluggable cost and constraint models.

DOUBTING

Deflated Sharpe at the number of trials, the probability of backtest overfitting, purged combinatorial cross-validation splits, the reality check, comparison against random entries, trade clustering, parameter decay, regime-stratified performance and the break-even cost a strategy can bear.

> A Sharpe ratio is a claim about one path. The trial count, the cost it survives and the paths it did not take are what decide whether to believe it.

11 Portfolio, allocation and finance

Three runtimes turn a view into capital. Portfolio (23 tools) builds and costs positions; allocation (10) splits capital across return streams; finance (9) prices what holding a book costs and what a stream of cash is worth.

RUNTIME	WHAT IT HOLDS
`portfolio`	Covariance with diagnostics; an exact constrained optimizer (minimum variance, maximum utility, maximum Sharpe, tracking error) under bounds, groups, factor-exposure limits and risk caps, with a KKT certificate and shadow prices; the efficient frontier; risk parity, hierarchical risk parity and maximum diversification; factor and tail risk decomposition; stress tests, capacity, trade costs and position sizing
`allocation`	Streams on one calendar; allocation across sleeves with bootstrap weight bands; a walk-forward test of the rule against equal weight; choosing among rules as you go; registered models as sleeves; Kelly with its uncertainty; volatility-target and CPPI overlays; capacity curves; Brinson attribution; time- and money-weighted book returns
`finance`	Financing and borrow; Reg T and FINRA margin and the runway a stress path leaves; currency exposure; carry; the FX forward by covered interest parity; NPV, every IRR, duration, cost of capital and DCF

The rebalance, net of trading costs

`optimize_rebalance` finds the trade to make from the weights held, with linear and quadratic costs in the objective. Names inside their no-trade band keep their weight exactly, and the frictionless target is reported beside the trade, so the gap shows what the costs held back. Two hundred assets solve in about 25 ms.

Choosing among rules as you go

`select_allocation_rule` picks at each rebalance from each rule's own out-of-sample record before that date, and reports the hindsight pick beside it. On planted data it kept 0.28 of a 0.37 Sharpe edge, cost 0.02 against equal weight with no edge, and never claimed one. An optional regime condition added 0.050 Sharpe to a volatility-targeted risk parity rule, ahead in 28 of 30 histories.

12 Derivatives, delta one and microstructure

Three runtimes answer what an instrument is worth, which instrument is the cheapest way to hold an exposure, and what the market will charge to trade it.

DERIVATIVES · 13

Pricing and greeks to second order, implied volatility, multi-leg payoffs, smile and term-structure fits with arbitrage checks, put-call parity, the expected move, delta-hedge simulation, full revaluation grids, and an option book's dollar greeks with the multiplier required. Quotes are arguments: the library fetches no option chains.

DELTA ONE · 18

Carry and basis against a quoted future, curves and roll costs, beta translated into contracts and whether the hedge worked, baskets and index rebalances, ETF fair value, total return swaps and futures. The holding horizon can reverse which instrument is cheapest.

MICROSTRUCTURE · 17

Four tools measure spreads and order flow from ticks and refuse without a tick feed; seven estimate the same quantities from OHLCV, each saying what it is a proxy for. `check_spread_proxy` turns a proxy's error on one name into a number.



PART IV

Evidence

What a decision record proves, what the audits found, how the tests are built to notice a defect, how fast the extension measured, and the results that were measured and did not help.

13 The audit trail

14 The correctness audits

15 The testing regime

16 Performance, measured

17 Where it did not help

13 The audit trail

A call routed through `dispatch()` can write an immutable decision record: its inputs, the market data it pulled with content hashes, which execution path ran, and a hash of its output. That is enough to tell a stale or tampered cache from a genuine code change.

1 · CALL
A tool runs through `dispatch()` with validated inputs.

2 · RECORD
Inputs, data hashes, the path taken and the output hash go to the day's JSONL file.

3 · CHAIN
Each record hashes the one before it, across calendar days, so a deleted day is detected.

4 · ANCHOR
Optionally, an Ed25519 checkpoint signs a day's chain endpoint.

5 · REPLAY
`sqt replay` re-runs the call and says whether it still reproduces.

THE SQT COMMAND

```
sqt replay <request_id>              # re-run it: do the data and the output still match?
sqt compare <id_a> <id_b>            # diff two records, field by field
sqt verify [--file PATH]             # check the hash chain, whole trail or one file
sqt anchor <date> --key PATH         # sign a checkpoint on a day's chain endpoint
sqt export --start D --end D --out F # a date range, its manifest and a standalone verifier
```

EXIT CODES THAT MEAN SOMETHING

`sqt replay` exits 0 when the output reproduced exactly, 1 on a confirmed mismatch, 2 when the record has no output hash to compare against, and 3 when it reproduced to twelve significant digits on a different native build or instruction-set path. `sqt verify` exits 0 when the chain is clean.

WHAT IT DOES AND DOES NOT CERTIFY

It is an engineering control: tamper *detection*, not tamper prevention or regulatory certification. Checkpoint signing closes the one gap the chain cannot close alone, an internally consistent rewrite of a whole day. An exported bundle carries a dependency-free verifier for an auditor who does not install the package, and named input fields can be redacted to a salted, non-reversible hash before a record is written.

14 The correctness audits

Both tiers have been through line-by-line audits. Most findings were not exotic: they were ordinary code that looked right, and the reason each survived review is usually more instructive than the fix.

AUDIT	FINDINGS	THE SHAPE OF WHAT IT FOUND
The core, line by line	41	31 in the Python tier, 10 in C++. Five backend divergences, where the same call answered differently with the extension built; checks that lived only in the C++ branch; two JIT kernels that could read past their arrays on short input.

AUDIT	FINDINGS	THE SHAPE OF WHAT IT FOUND
The modeling runtime, first review	7	Critical, two of them look-ahead channels: walk-forward was given an embargo but never the target horizon, and a negative lookback let a feature read future prices while its point-in-time label stayed satisfied.
The modeling runtime, second review	20	Results that were plausible, consistent and wrong: a full-refit model that had seen prices past its own cutoff, an end date that meant different things per provider, a cross-section that could mix dates.
The live-data pass, 20 September 2026	20	The first run against a real vendor tape, which found defects no synthetic fixture could show.

Further passes over the portfolio, screener and agent tools, and a full-codebase review, take the record to ten sections. None of it was catchable by the suite as it stood. Every finding was reproduced against a live interpreter before it was fixed, and each is pinned by a regression test.

> **WHY PARITY TESTS WERE NOT ENOUGH**
>
> A test that pins only one backend cannot see the two disagree, which is how several divergences survived. The fixes are pinned by tests that hold the two backends against *each other*, and against a second oracle that neither of them shares.

15 The testing regime

Nine layers, each catching a class of defect the others cannot. The rule underneath all of them: test against an answer known by construction, never against a recorded output.

LAYER	WHAT IT HOLDS	WHAT IT HAS CAUGHT
1 · Planted answers	Greeks against finite differences, parity against the model's own prices, risk parity against its closed form	A test asserting a recorded number passes forever and catches nothing; this one catches the formula
1b · A second oracle	Every C++ kernel checked against an independent reference, not only against its Python twin	Two backends agreeing on the wrong answer, which a parity test cannot see
1c · Extension off	The whole suite run with the compiled extension switched off	Fallbacks reachable only behind a monkeypatch, easily mistaken for dead code
2 · Whole surface	Invariants across every tool: registered in all six places, schema and dispatch agree	A tool registered halfway, which no per-tool test can catch
3 · Fuzzing	Schema-synthesized adversarial inputs: a named refusal or a valid result, never a crash	Six real bugs on its first run, three of them index errors on short data
3b · Metamorphic	Relations that say two inputs must give the same answer	A function that is consistently wrong for every input
4 · Determinism	The same arguments give identical output; a different seed gives a different answer	A `seed` silently ignored
5 · Documentation	Generated indexes compared with the registry; every count in prose checked	85 of 157 tools in no document, and a README advertising 95 tools in six runtimes

LAYER	WHAT IT HOLDS	WHAT IT HAS CAUGHT
6 · Mutation	A catalogue of deliberate defects, each of which the suite must notice	A false-positive test whose bar the uncorrected rate already slipped under

19,256
Python tests; 19,084 pass and 94 skip with the extension built, the rest need network

95,816
assertion-level checks in 16 C++ test executables, 50,663 of them from the fuzz harness

21 / 21
mutations killed in the current catalogue

What none of this establishes is that the numbers are useful. The regime checks that a statistic is computed correctly, refuses cleanly, reproduces exactly and is documented honestly. Whether a strategy makes money is not a property a test can assert.

16 Performance, measured

The C++ extension is optional, and the API is identical either way. Every figure is measured by toggling the same module's `HAS_CPP` flag and timing both paths back to back, and the ports that did not pay off are reported with the ones that did.

An extension built from other C++ sources than the ones beside the package is refused at import, with a warning naming the file and the command that rebuilds it, so it cannot answer with old kernels.

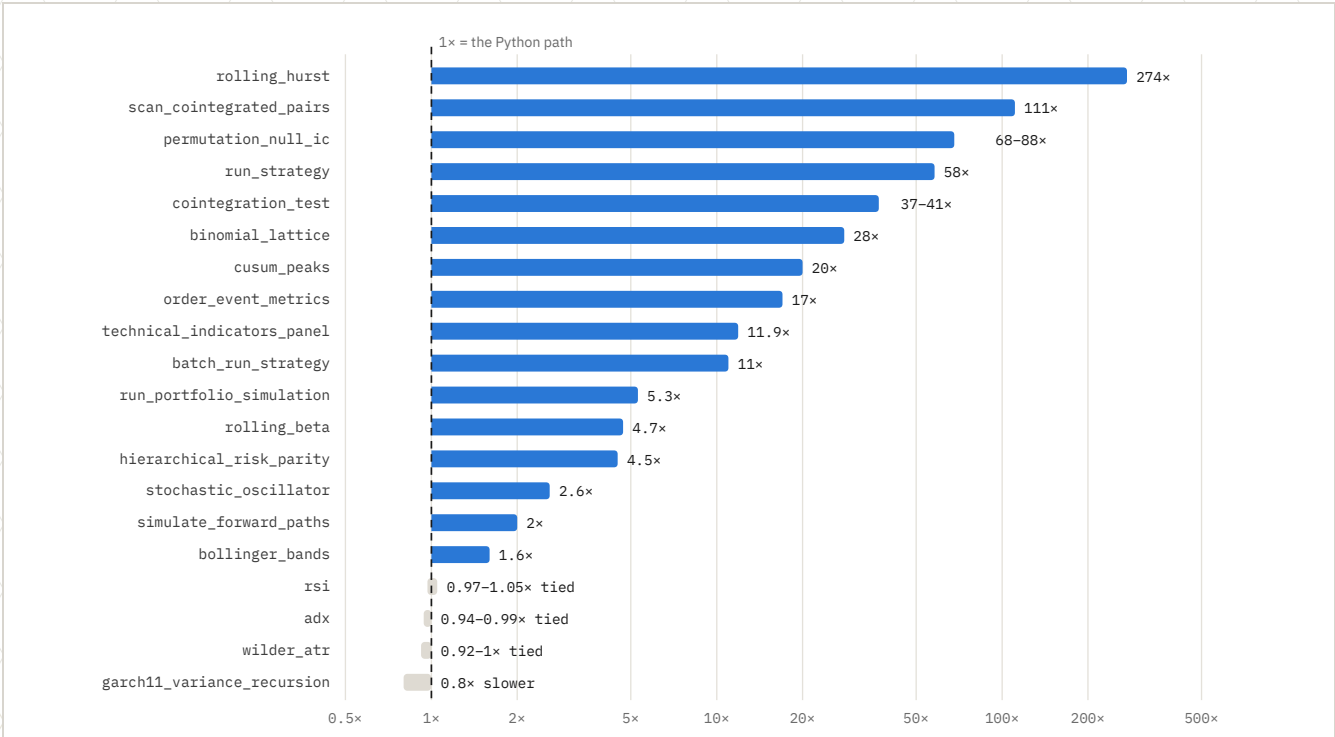


Figure 2. The C++ path against the Python path it replaces, the same call on the same machine, log scale. A range is the spread over the sizes measured. Grey bars are ties.

WHERE IT PAYS

Rolling Hurst at 274×. A 2,000-name cointegration scan, both orders of all 1,999,000 pairs, in 1.7 minutes at 500 bars. The backtest kernel at about 58×, once a wrapper-redundancy bug that had held it at 1.0× end to end was found and fixed.

WHERE IT TIES

RSI, ADX and Wilder's ATR tie with the warm JIT-compiled Python path, and the GARCH recursion is slightly slower, at 0.8×. Those ports exist for the 200 ms to 1.1 s compile on the first call in a fresh process, which they remove, not for steady-state speed.

17 Where it did not help

An option that is measured and does not help stays off by default, and the measurement is published beside the ones that worked. These are the results that decided defaults.

WHAT WAS TRIED	WHAT IT MEASURED	WHAT IT DECIDED
Kelly sizing of model predictions	The gross limit sized every Kelly book, so a quarter, a half and full Kelly gave the same Sharpe at about twice the drawdown; its +0.07 lead is within 1.5 standard errors	Opt-in, quarter Kelly when chosen
Shrinking the information coefficient	Cost 0.14 ± 0.05 Sharpe with an edge, and 0.24 ± 0.07 where correlated names score alike	Off by default
Optimizer sizing against damped rank	A tie with a real edge, +0.01 ± 0.05	Reported as a tie, not a win
Choosing the rule as you go, without an edge	−0.023 ± 0.013 Sharpe against equal weight; on regime data it did not find the better rule, a hindsight gap of 0.18	Ships with the hindsight pick reported beside it
Regime conditioning on max Sharpe	Not distinguishable from zero, where risk parity gained 0.050	Opt-in, per rule
Screening factors against keeping all	The selected model's book was only 0 to 2.3 bp a year better than the model with every candidate	Said in the result, not hidden
Porting RSI, ADX, ATR and GARCH to C++	Tied with the warm JIT path, GARCH slightly slower at 0.8×	Kept for the cold start, and labelled as such

The planted-data figures are paired comparisons over many simulated histories, net of costs, with the standard error of the difference. They show what a rule does when the truth is known, not what it will earn.



PART V

Using it

Install it, call it from Python or serve it over MCP, and know where its edges are before relying on it.

18 Getting started

19 Limits, stated plainly

18 Getting started

Python 3.10 or later. `pip install .` builds the C++ extension when a toolchain is present and installs the pure-Python package when it is not; every tool works either way.

```
pip install .                              # the library, with the extension where it can build
pip install "standard_quant_tools[mcp]"    # adds the sqt-mcp server
pip install "standard_quant_tools[signing]"  # adds Ed25519 checkpoint signing
pip install "standard_quant_tools[polars]"   # adds optional polars interop
```

FROM PYTHON

```
from standard_quant_tools.agent.tools import dispatch, get_agent_tools

schemas = get_agent_tools()      # the analysis facade's function-calling schemas
risk = dispatch("analyze_stock_risk", {"symbol": "NVDA", "period": "1y"})
```

`dispatch` validates the arguments against the tool's input model and returns its result as a dictionary. An unknown argument is refused, not ignored.

OVER MCP

```
sqt-mcp --runtime research       # serve one runtime to the client
sqt-mcp --runtime backtest
sqt-mcp --print-budget           # what each runtime and category costs a client's context
```

Any MCP client launches `sqt-mcp` with the runtime it should be served. Thinned schemas are the default, and `describe_tool` returns any of them in full.

Configuration and secrets

Provider settings live in a `.env` file that is never committed, or in a CI system's encrypted secrets under the same names; `load_env()` is a harmless no-op when there is none. The audit trail writes where `SQT_AUDIT_DIR` points, and `SQT_AUDIT_ENABLED=0` turns record writes off.

Where to read next

QUESTION	GUIDE IN THE REPOSITORY
Every module's API in one place	`Documentation/00_module_reference.md`
Every tool, with the description the model sees	`Documentation/20_tool_index.md`
Why runtimes, and how references cross them	`Documentation/19_runtimes.md`
Serving the library over MCP	`Documentation/18_mcp.md`
What a decision record can certify	`Documentation/10_auditability.md`
Every measured speedup, and the ports that did not pay	`Documentation/16_performance.md`
The nine test layers and what each caught	`Documentation/25_testing.md`

19 Limits, stated plainly

What the library does not do, or does only partly. Each limit is also stated where it bites, in the tool's own description or refusal.

LIMIT	WHAT IT MEANS IN PRACTICE
No option-chain data	The derivatives tools take quotes as arguments. Nothing in the library fetches a chain, and no description implies otherwise.
The whole surface does not fit a session	727 KB of schema is about 186,000 tokens. A client is served one runtime, by design; serving all twelve is a diagnostic.
An audit trail is not certification	It detects tampering; it does not prevent it, and it is not a regulatory attestation. `sqt keygen` is for local development; a real deployment signs through its own key service.
In-sample is in-sample	An allocation's reported Sharpe, a factor model's fit and a full-refit model's scores are in-sample, and are labelled so. Out-of-sample figures come from the walk-forward tools.
pandas is required	Polars interop covers a growing subset of functions; pandas stays the default and required backend.
Live vendor behaviour	The suite runs on planted and recorded data. What a vendor's live feed does is established only with a live key, in the tests marked as needing one.
Correct is not useful	The tests show a statistic is computed correctly and reproduces. Whether a strategy built on it makes money is not something any test, or this document, asserts.

NOT INVESTMENT ADVICE

Standard Quant Tools is a software library. Nothing in it, or in this briefing, is a recommendation to buy or sell anything, and the planted-data results in Part IV describe what a method does when the truth is known, not what it will earn.



APPENDICES

Appendices

Every figure in this document and where it comes from, the terms it uses, and every tool, generated from the registry.

A	Numbers at a glance
B	Glossary
C	The complete tool catalog

A Appendix A · Numbers at a glance

Every figure here was measured from the code on 27 October 2026, at commit `09727c7`, unless the row says where else it comes from. Re-measure rather than edit.

FIGURE	VALUE	SOURCE OR METHOD
Tools in the registry	266	`all_runtimes()`
Runtimes · categories	12 · 15	`all_runtimes()`, `TOOL_CATEGORY`
Tools in the analysis facade	217	`get_agent_tools()`
Whole-surface schema, full detail	744,271 B	`sqt-mcp --print-budget`; ~186,067 tokens
Most expensive runtime	218,231 B	modeling, 29% of the total
Average tool schema over the wire	3,132 B	Documentation/18_mcp.md
Tools fitting 180,000 bytes, smallest first	127	Documentation/18_mcp.md
Default MCP session	62 tools	four categories, ~23,000 tokens
Python tests · passing · skipped	19,256 · 19,084 · 94	with the extension built; 78 integration tests need network
C++ test executables · checks	16 · 95,816	ctest; 50,663 of the checks from the fuzz harness
Mutations killed	21 of 21	`scripts/mutation_testing.py`
Audit findings, core	41	31 Python, 10 C++; Documentation/17_correctness.md
Modeling review findings	7 + 20	Documentation/17_correctness.md
Live-data pass findings	20	20 September 2026, against a real vendor tape
Rolling Hurst speedup	274×	n = 2,000, window 200; Documentation/16_performance.md
2,000-name pair scan	1.7 min	both orders of 1,999,000 pairs at 500 bars
Backtest kernel speedup	~58×	`run_strategy`, n = 2,000
Rebalance solve, 200 assets	~25 ms	`optimize_rebalance`
Modeling features · estimators · targets	31 · 27 · 18	the modeling registries; 8 estimators optional
Lines of Python · C++ · tests	~198k · ~14.6k · ~193k	the package, the extension and the Python tests
Commits	595	since 15 February 2026
Noise factors selected	0 of 4,000	planted data, `n_shifts` 500
Rule selection, with an edge	0.28 of 0.37	Sharpe kept, planted data, net of costs
Regime option on risk parity	+0.050	Sharpe, ahead in 28 of 30 histories

B Appendix B · Glossary

TERM	MEANING HERE
Runtime	An execution boundary with its own dispatch table. A tool from another runtime is unroutable through it.
Category	A narrower grouping within a runtime, used to scope a session further.
Analysis facade	The 217 tools `get_agent_tools()` returns, routed by `dispatch()`.
Reference	An `sqt://kind/run_id/name` string standing for a published bulk value, its kind checked on resolve.
Decision record	One JSONL entry per dispatched call: inputs, data hashes, execution path, output hash, chained to the one before.
Checkpoint	An Ed25519 signature over a day's chain endpoint, verifiable with the public key alone.
Temporal contract	What a source can promise about when its facts became knowable, asked before fetching.
Bundle	Several frames and the contract each was fetched under, validated as one unit.
Purging · embargo	Dropping training rows whose labels overlap the test period, by label end date, and a gap after it.
Deflated Sharpe	A Sharpe ratio corrected for the number of strategies tried and for non-normal returns.
PBO	The probability of backtest overfitting: how often the in-sample winner ranks below the median out of sample.
Thinning	Serving a shortened schema for an expensive tool, with `describe_tool` returning the full one.
HAS_CPP	Each module's flag for whether its C++ kernel is in use; toggled to time both paths of the same call.
Planted answer	A test input built so the right output is known by construction, rather than recorded from a past run.
Metamorphic relation	A rule saying two different inputs must give the same answer, which catches a consistently wrong function.
Mutation testing	Planting a deliberate defect and requiring the suite to fail, which shows the tests would notice one.

C Appendix C · The complete tool catalog

All 266 tools on 27 October 2026, generated from the live registry rather than written by hand, grouped by the runtime that owns them, largest first. Each description is the first sentence of the text the model reads when it chooses the tool.

A runtime is an execution boundary: each holds its own dispatch table, so a tool from another runtime is unroutable, and the refusal names the runtime that owns it. 217 of the 266 also sit in the analysis facade, `get_agent_tools()`. The 2 marked **LONG** are long-running, and an MCP session serves them only with `--enable-long-running`.

`research` · Research · 46 tools

Describe an asset or a universe: screen it, profile its risk and technicals, and analyze its statistical structure (factors, cointegration, PCA, Hurst, correlation).

`analyze_stock_risk`	Full risk profile of one asset against a benchmark: alpha, beta, Sharpe, VaR and CVaR in a single call.
`analyze_tail_dependence`	Whether two assets move together IN THE TAIL, which is the only regime a diversification claim has to survive.

backtest_var_forecasts	Whether a series of VaR and ES forecasts was right against the returns that followed: Kupiec, Christoffersen, the Basel traffic light and Acerbi-Szekely Z2.
calculate_series_metrics	Risk and return metrics for ANY return series -- a symbol, an `sqt://` reference from another runtime, or values passed inline.
compare_distributions	Whether two samples came from the same distribution -- not whether their means differ.
compute_indicator_panel	Indicator HISTORY for a whole universe, published one `sqt://` reference per indicator.
decompose_returns	Where compound growth actually came from.
detect_change_points	When the process generating a series CHANGED, by binary segmentation on the mean, and separately on the variance.
detect_regimes	Label each observation with a volatility regime, by a Gaussian mixture.
estimate_factor_model	A factor model from return series: each asset's OLS loadings on the caller's factor returns, the factor covariance F and the specific variances D, with standard errors.
estimate_tail_index	How fat the tail is, by the Hill estimator.
forecast_var_series	One-step-ahead VaR and expected shortfall for every date of a return stream, each from the rolling window before it, and the forecast for the next period.
get_advanced_indicators	Compute Parabolic SAR (trend), Wilder ATR (volatility), and MFI (volume-flow oscillator).
get_bootstrap_interval	A confidence interval for a statistic, by BLOCK bootstrap.
get_correlation_analysis	Correlation matrix, avg pairwise correlation, most/least correlated pair, and diversification ratio for a universe.
get_correlation_stability	Whether a correlation is a property of the pair or an average over two different regimes.
get_data_quality_report	Dataset provenance plus missing-bar/stale-price/price-jump/thin-volume detection and duplicate-timestamp, out-of-order and OHLC-consistency checks on a symbol's OHLCV.
get_drawdown_profile	Every drawdown, not just the worst.
get_entropy_measures	How predictable a series is WITHOUT assuming the predictability is linear.
get_extended_risk_metrics	Extended risk: Calmar ratio, Treynor ratio, parametric VaR 95/99, historical VaR 99, CVaR 99.
get_lead_lag_matrix	Which series move first across a universe -- and why the answer is usually noise.
get_partial_correlation	The correlation between two assets once the common drivers are removed from BOTH.
get_portfolio_analysis	Risk and return metrics for a basket held at fixed weights: portfolio volatility, correlation structure, and contribution by position.
get_rally_signal	Detect a rally via 5 confirming signals: return z-score, ADX trend strength, DI+/DI- direction, Hurst trending regime, and new-high breakout.
get_rolling_beta	Compute rolling OLS beta to detect beta drift over time vs a benchmark.
get_sharpe_stability	Whether the edge DECAYED, or the full-sample Sharpe is the average of a good period and a dead one.
get_stock_fundamentals	Fetch company metadata and key financial ratios (PE, P/B, debt/equity, ROE, market cap).
get_tail_risk_metrics	Extreme Value Theory tail risk (Peaks-Over-Threshold GPD fit): VaR/CVaR extrapolated from the fitted tail, compared against the naive historical quantile.
get_technical_analysis	Technical indicators for ONE ticker, with the parameters you choose -- RSI, MACD, Bollinger, moving averages and the rest.
get_technical_panel	Indicators (RSI/ADX/ATR/Bollinger/Stochastic) for a whole ticker universe in one native call, reported at the latest bar.
get_volatility_estimators	Realized volatility via Parkinson, Garman-Klass, and Yang-Zhang estimators vs.
run_cointegration_test	Engle-Granger cointegration: hedge ratio, half-life, spread z-score signal.
run_factor_regression	Multi-factor OLS regression: alpha, loadings, t-stats, p-values, R^2.
run_garch_volatility_forecast	GARCH(1,1) conditional volatility: fits how variance evolves over time and forecasts it forward, unlike get_volatility_estimators' backward-looking realized estimates.
run_hurst_analysis	Hurst exponent (DFA, or R/S corrected for its small-sample upward bias), with regime classification against a random-walk band that widens for short series (regime_band).

run_kalman_hedge_ratio	Time-varying hedge ratio via a Kalman filter — a staleness diagnostic companion to run_cointegration_test's static OLS hedge ratio.
run_pca_analysis	PCA on multi-asset returns: explained variance, loadings, factor contributions.
run_screener	Filter a stock universe by fundamental and technical criteria.
run_seasonality_analysis	Whether performance concentrates in a day of the week or a month of the year, corrected for having looked at all of them.
run_stationarity_tests	ADF, KPSS and the variance ratio, with the four-way verdict spelled out.
scan_pairs LONG	Scan a ticker universe for cointegrated pairs, ranked by half-life.
screen_factor_candidates	Which candidate factors earn a place in a risk model, out of sample: incremental R-squared against circular shifts, loading stability, VIF and minimum-variance risk.
test_autocorrelation	A JOINT Ljung-Box test for autocorrelation across lags, rather than one test per lag.
test_granger_causality	Whether one series helps predict another beyond that series' own past.
test_normality	Whether a return series is normal -- and it is not, which is the point.
test_structural_break	A Chow test for a break at a KNOWN date.

modeling · Modeling · 38 tools

Build, validate and score a statistical model from this library's own features: dataset construction, leakage-purged walk-forward fitting, the model registry, and evaluation of out-of-sample predictions.

analyze_features	Score a built dataset's FEATURES before fitting anything: coverage, turnover, cross-sectional IC and ICIR, decile spread and monotonicity, and near-duplicate features.
analyze_model_errors	WHERE a registered model is wrong, not merely how wrong on average.
attach_model_outcomes	Join predictions to the outcome they were predicting and publish the result -- the step that makes this library's own output scoreable.
attest_model_package	Is a registered package still the one that was registered, and did a key anybody trusts say so.
backtest_model_signal	Turn a registered model's out-of-sample predictions into a tradeable signal panel and publish it.
build_model_dataset	Fetch OHLCV, compute requested features/target, persist the panel.
build_model_ensemble	Combine several registered models into one prediction series, and publish it as an `sqt://predictions` reference the backtest bridge reads like any other.
check_leakage	Ask whether a set of features is temporally safe to fit on — before building a dataset with them.
compare_models	Rank registered models side by side on their out-of-sample metrics, or -- with method='paired' -- test whether one is actually better than a reference.
compare_signals	Decide whether one signal actually beat another, and correct a family of p-values for having asked more than once.
describe_estimator	What an estimator's params actually accept: every parameter name, its type, its range, its exact choices and the reason behind each bound.
describe_exchange_calendar	List the exchange calendars this installation knows, and resolve one into the numbers that annualize an intraday model.
estimate_feature_warmup	How many bars of history a feature spec burns before its first usable row, at the parameters you are actually requesting.
evaluate_model_portfolio	Evaluate a model's out-of-sample predictions as a shared-cash portfolio: predictions become target weights, simulated with costs.
evaluate_predictions_portfolio	Evaluate ANY published predictions reference as a shared-cash portfolio: the predictions become target weights, simulated with costs.
explain_dataset_row_loss	Which column cost which training rows, and which are free to drop.
inspect_dataset	Describe one dataset built by build_model_dataset or registered by register_external_panel -- the counterpart of inspect_model.
inspect_model	Inspect a registered model's summary/importance/validation/lineage/provenance.
join_point_in_time	Attach point-in-time records to a built dataset, each panel row getting the most recent record AVAILABLE by then.

list_datasets	Every built dataset panel, newest first, with row/entity/feature counts and date span.
list_features	Which features this library can build, what each one measures, and what it costs to compute.
list_modeling_capabilities	What this modeling runtime can do: tasks, estimators and their capabilities, features, target types, validation schemes, preprocessing and weighting options.
list_models	Every registered model, newest first, with its task, estimator, headline out-of-sample metric and source dataset.
list_remote_models	List the model ids in an artifact store -- a directory, or a bucket reached through fsspec -- so a package registered on another machine can be found before it is pulled.
monitor_model	Feature and prediction drift of a scored universe against the model's training reference (PSI and KS per feature).
plan_model_experiment	The schedule run_model_experiment would execute, on the real panel, before it executes any of it: every fold with its train and test dates.
predict_survival_curve	Read a registered SURVIVAL model's curve S(t | x) for each entity as of a date: how likely each one is to still be waiting at t, plus the median time until its event.
preview_preprocessing	Fit a preprocessing pipeline on a sample of a built dataset and report what it does to the columns, before an experiment runs it on every fold.
preview_sample_weights	What a WeightingSpec would actually do to the training rows, before a model is fitted under it.
promote_model	Record a lifecycle decision for a registered model, with a reason and the evidence it rests on.
pull_model_package	Register a model package from an artifact store into THIS runs directory, verified on the way in.
register_external_panel	Register a feature matrix computed OUTSIDE this library -- by a C++ pipeline over an L2 feed, a warehouse query, another system -- as a modeling dataset, without copying it.
run_model_experiment	Fit + walk-forward validate + register a model from a persisted dataset.
score_model	Run a registered model forward and get its predictions for a universe as of a date.
score_prediction_intervals	Score the quantile and interval columns of a published predictions reference against the realized outcome: pinball loss per level, the quantile crossing rate, coverage AND...
score_predictions	Score a predictions reference against its realized outcome — accuracy metrics, cross-sectional IC and ICIR, a predict-the-mean baseline and the headline test (`beats_null`).
validate_model_spec	Check a ModelSpec before spending an experiment on it: that the estimator exists for the task, that its parameters are accepted, and how many fits the spec implies.
validate_pit_records	Check point-in-time records BEFORE joining them onto anything.

backtest · Backtest · 35 tools

Run, optimize, validate and diagnose a trading strategy — the library's built-in ones or a signal the caller computed themselves.

analyze_parameter_decay	Whether performance degrades SMOOTHLY as a parameter moves or falls off a cliff.
analyze_trade_clustering	Whether wins and losses arrive in RUNS, measured in the original order.
build_purged_cv_splits	Train/test index sets that do not leak, for a label that looks forward.
compare_against_random	Whether the strategy beats a coin that traded the same instrument the same number of times.
compare_cost_models	Run one strategy under several cost assumptions on a single fetched signal series, and solve for the commission rate at which its total return reaches zero.
compare_strategies	Run all four strategies on the same symbol and return ranked results vs buy-and-hold.
estimate_backtest_overfitting	PBO: how often the configuration that wins in-sample loses out-of-sample, across every equal split of the period.
estimate_break_even_cost	The per-trade cost at which this edge disappears -- the number every backtest should report and almost none do.
get_backtest_diagnostics	Extended diagnostics for a built-in strategy: top drawdown episodes, trade expectancy/payoff/streaks with MAE/MFE, and exposure stats.
get_deflated_sharpe_ratio	The probability a Sharpe ratio is real GIVEN how many strategies were tried.
get_drawdown_table	Every drawdown episode in a persisted equity curve (peak, trough, recovery, depth, duration), deepest first, from an equity_curve_uri rather than by re-running the backtest.

get_exposure_attribution	How much of the return came from being RIGHT versus from being INVESTED.
get_regime_stratified_performance	Performance broken out by regime, because one Sharpe over a mixed sample describes none of them.
get_robustness_diagnostics	Same-sample robustness checks for a grid search: parameter sensitivity, Deflated Sharpe Ratio, and a block-bootstrap confidence interval on the best trial's Sharpe ratio.
run_backtest_compact	Compact backtest result: summary/risk/exposure/cost sub-reports plus equity-curve/trade-log artifact URIs, instead of embedding the full data inline like run_sma_backtest etc.
run_backtest_optimization LONG	Grid-search strategy parameters and return the top N combinations ranked by a chosen metric.
run_bollinger_backtest	Backtest a Bollinger Band reversion rule: buy the lower band, sell the upper.
run_buy_and_hold	Buy-and-hold baseline: long the full period.
run_custom_signal_backtest	Backtest a signal computed outside this library (your own alpha model) on one symbol.
run_futures_backtest	Simulate a FUTURES account, whose books the shared-cash engine cannot keep.
run_futures_hedge_backtest	Carry a cash book and its futures hedge together, bar by bar, and report the two P&L streams SEPARATELY.
run_macd_backtest	Backtest a MACD signal-line crossover.
run_monte_carlo_simulation	Monte Carlo forward simulation of a portfolio's future equity paths via moving-block bootstrap of its historical returns.
run_monte_carlo_trade_paths	The distribution of outcomes the same edge could have produced, by RESHUFFLING the trades.
run_pair_trade_backtest	Backtest a cointegrated pair as one synchronized two-leg trade — both legs enter/exit together and share one cash account, unlike scan_pairs which only screens candidates.
run_portfolio_simulation	True shared-cash portfolio simulation with rebalancing at target-weight dates.
run_reality_check	Is a result better than luck after a search, under the hypothesis named in question.
run_regime_adaptive_backtest	Classify market regime via Hurst, auto-select and optimise the best strategy.
run_regime_adaptive_walkforward_backtest	Leakage-free regime-adaptive backtest: regime/strategy/parameter selection per walk-forward window, evaluated strictly out-of-sample.
run_rsi_backtest	Backtest an RSI mean-reversion rule: buy oversold, sell overbought.
run_signal_panel_backtest	Backtest a pre-computed signal panel across a ticker universe, combined into portfolio metrics.
run_sma_backtest	Backtest a moving-average crossover: long when the fast average crosses above the slow one, flat or short when it crosses back.
run_strategy_matrix	Every requested strategy against every requested ticker in one call, ranked.
run_terminal_monte_carlo	Monte Carlo that keeps only where the paths ENDED, so the simulation count is capped by wall-clock rather than by memory.
run_walk_forward_backtest	Walk-forward validation: optimise in-sample, evaluate out-of-sample, return OOS stats.

meta · Discovery & Provenance · 25 tools

Questions about the library and the session rather than about a market: what this library accepts and what the data provider can serve, and what a past tool call did and whether it still reproduces.

compare_artifacts	A field-by-field diff of two result objects, ordered by the size of the change.
compare_data_sources	Fetch the same fundamentals from two providers and report where they disagree, separating a SCALE difference from a DEFINITION difference from noise.
compare_decisions	Diff two recorded calls — tool, inputs, output hash, git commit — and say which of the candidate causes the evidence supports.
convert_reference	Turn one kind of published value into another and publish the result: raw model predictions into a signal panel, scores into portfolio weights.
describe_artifact	Shape, date span, per-column statistics and both ends of a persisted Parquet artifact, by URI or by the store key list_artifacts reports.
describe_audit_log	What the decision log holds and what it is configured to do: which dates, how many records, how large, and its recording, redaction, retention and signing settings.

describe_data_capabilities	What a data provider can serve — tick trades, top-of-book quotes, L2 depth, order events, point-in-time records, async OHLCV and supported intervals.
describe_effective_config	Every SQT_* setting this process reads, resolved through the functions that read it rather than echoed from the environment, so an unset variable reports the value in force.
describe_numeric_contract	The numerical rules every public boundary in this library enforces, with the threshold each bites at, an excerpt of the message it raises, and why the line is where it is.
describe_reference	What a handoff reference points at — its content kind, shape, date span and which runtime published it.
describe_runtime	What each runtime is for, which categories it owns, and which tools it holds.
describe_temporal_contract	What a data source can say about WHEN its facts became knowable, asked BEFORE fetching anything.
describe_tool	One tool's full contract — arguments, result fields, owning runtime, and whether calling it fetches data or writes an artifact.
estimate_tool_cost	What each runtime costs a client's context, in bytes and approximate tokens, before any call is made.
explain_decision	What one recorded tool call did: its inputs, the market data it read with the content hashes those inputs had at the time, which execution path ran (C++/Numba/Python) and...
export_audit_bundle	Package a date range of the audit log plus its chain index, any checkpoint sidecars and a manifest into one zip.
find_decisions	Search the decision log by tool, status and date, and get back the request ids explain_decision, replay_decision and compare_decisions take.
list_artifacts	Every artifact this library has persisted, or one run's: key, absolute URI, size, last-modified time and -- on request -- the content hash.
list_reference_kinds	Every content kind a handoff reference can carry and what converts to what — the map of which producer outputs can reach which consumer inputs.
list_strategies	Every built-in strategy's parameter contract: names, kinds, defaults, bounds and cross-parameter relations.
list_stress_scenarios	The named historical crash windows run_stress_test accepts, with each window's dates.
read_reference	The actual values at chosen rows of a handoff reference — by date, or the first/last N.
replay_decision	Re-run a recorded call and classify the result as reproduced, reproduced_to_12_digits, data_changed, data_undecided, code_changed or not_comparable.
validate_tool_call	Check arguments against a tool's schema WITHOUT calling it, including the strategy parameter contract that the JSON schema cannot express.
verify_audit_integrity	Check the audit log's tamper-evident hash chain, for one day or the whole trail, optionally including that day's Ed25519 checkpoint signature.

portfolio · Portfolio & Execution · 23 tools

Turn a view into a position and price what it costs.

analyze_concentration	How concentrated a portfolio is, in numbers with known interpretations.
construct_weights_from_scores	Turn alpha scores into portfolio weights and STOP, so the weights can be looked at before anything is simulated.
decompose_factor_risk	Splits a portfolio's risk under a factor model (B F B' + D) into factor and specific parts, by factor and by asset, with tracking error against a benchmark split the same way.
decompose_tail_risk	Which positions a portfolio's expected shortfall and VaR come from: Euler component ES and kernel-smoothed component VaR from a scenario matrix or a returns_ref.
estimate_covariance	A covariance matrix plus the diagnostics that say whether to trust it.
estimate_trade_cost	Itemized cost of one hypothetical trade under a composed cost model: commission, spread, square-root market impact, short borrow and margin interest.
get_capacity_report	How much account size a target-weight portfolio can support before positions become too large relative to each ticker's own trading volume.
get_efficient_frontier	The WHOLE efficient frontier in one call, exactly: the closed form when weights are unbounded, Markowitz's critical line algorithm when long-only or capped.

get_factor_exposure_budget	What the portfolio is actually betting on, once the names collapse into factors.
get_liquidity_adjusted_var	VaR that accounts for not being able to exit at the mark.
get_liquidity_metrics	Amihud illiquidity ratio and Corwin-Schultz spread estimator per ticker — OHLCV-derived proxies for market depth and bid/ask spread.
get_marginal_risk_contribution	Where the risk in a portfolio you ALREADY hold is coming from, and what one more unit of each position costs.
get_portfolio_risk_attribution	Deep portfolio risk decomposition: MCR per asset, PCA attribution, optional factor model.
get_position_size	How large to trade, from a stop distance measured in ATR and an account risk budget.
optimize_hierarchical_risk_parity	Allocation that never INVERTS the covariance matrix.
optimize_max_diversification	The portfolio maximizing the DIVERSIFICATION RATIO: the weighted average of the assets' volatilities over the portfolio's own, 1.0 when all are perfectly correlated.
optimize_portfolio	Exact constrained portfolio optimization on expected returns and a covariance you supply: min variance, max utility, max Sharpe, min tracking error or max active utility.
optimize_rebalance	The trade to make from the weights you hold: exact mean-variance weights net of linear and quadratic trading costs, under turnover, gross and volatility caps.
optimize_risk_parity	Weights at which every asset contributes the SAME AMOUNT OF RISK -- not the same weight.
plan_rebalance	A day-by-day path from the weights you hold to the weights you want.
run_portfolio_optimization	Produce portfolio weights via Markowitz mean-variance (max_sharpe/min_volatility/target_return/target_volatility), risk parity, or Black-Litterman.
run_portfolio_scenarios	What a portfolio does under NAMED shocks rather than under a distribution.
run_stress_test	Replay a portfolio's weights against a named historical crash window (or custom date range) using real historical returns.

data · Data · 21 tools

Get the data and publish it as an `sqt://` reference every other runtime can read: OHLCV for one name or a whole universe, return panels, tick tapes and quote panels, provider guarantees, temporal contracts, and bundles that pair frames with what their sources promise.

build_continuous_futures_series	Stitch a chain of futures contracts into one continuous series, and publish TWO references rather than one.
build_data_bundle	Name several already-published frames as one unit and publish the manifest as a data_bundle reference.
compare_ratio_frames	Two providers' ratios side by side, with each disagreement CLASSIFIED rather than merely measured: a unit mismatch is fixable by rescaling, a definition difference is not.
describe_data_bundle	What frames a bundle names, how many rows and columns each has, and what each source can promise about revisions and point-in-time availability.
describe_external_dataset	What a registered dataset holds -- columns, dtypes, row count, depth levels, file count and size -- plus whether the file has changed since it was registered.
fetch_financial_ratios	Fetch a company's financial ratios and flag the ones that are implausible on their face -- a negative price-to-book, a dividend yield above a plausible ceiling.
fetch_ohlcv	Fetch one symbol's OHLCV bars and publish them as an `sqt://` price_panel reference rather than returning the rows inline.
fetch_ohlcv_panel	Fetch a whole universe's OHLCV in one call and publish it stacked long, with an `entity` column, as a price_panel reference.
fetch_order_book	Fetch L2 depth snapshots -- price and resting size at each level, level 1 the touch -- write them once under the run and return an order_book_panel reference.
fetch_order_events	Fetch order-by-order events -- every add, cancel, modify and fill with its own id -- write them once under the run and return an order_event_panel reference.
fetch_quote_panel	Fetch top-of-book quotes and publish them as a quote_panel reference, which is what signing trades by the Lee-Ready rule needs alongside a tape.
fetch_returns_panel	Fetch a universe and publish a wide date-by-ticker frame of returns as a returns_panel reference.

`fetch_tick_tape`	Fetch individual trades and publish them as a tick_tape reference, for the microstructure tools that measure rather than estimate.
`get_dataset_metadata`	What the active provider GUARANTEES about one symbol's data: whether prices are adjusted, the universe survivorship-free and values point-in-time, and which timezone stamps...
`infer_temporal_contract`	Read a published frame's own columns and report what they imply about when each row became knowable.
`preflight_vendor_request`	What a vendor request would cost and whether the data is even there -- asked for free, before the request is made.
`prepare_vendor_extract`	Convert a RAW vendor extract into this library's contract, the step BEFORE register_external_dataset.
`register_external_dataset`	Make a Parquet or CSV dataset already on your disk resolvable as an `sqt://` reference WITHOUT copying it, for data too large to fetch and republish.
`validate_data_bundle`	Whether a bundle is safe to model on, returned as a verdict with the blocking reasons rather than raised as an error.
`validate_external_dataset`	Scan a registered dataset in batches and report what would produce wrong numbers downstream, as a verdict with blocking reasons rather than an exception.
`validate_financial_ratios`	Check ratios you already hold -- from a vendor, a spreadsheet, another system -- for values that are implausible on their face, without fetching anything.

`delta_one` · Delta One · 18 tools

Which instrument is the cheapest way to own or hedge this exposure.

`analyze_basis_history`	Where today's basis sits inside its own history -- z-score, percentile, half-life and the distribution it came from.
`analyze_cash_futures_basis`	A quoted future against its carry-fair value, with the mispricing attributed to financing, dividend or borrow.
`analyze_dividend_points`	Index dividends as POINTS to a contract's expiry, dated and attributed by name.
`analyze_etf_fair_value`	An ETF's premium or discount, and what survives the cost of capturing it.
`analyze_futures_curve`	The futures term structure, and the FORWARD carry between expiries that a calendar spread actually prices.
`analyze_hedge_effectiveness`	Whether a hedge ratio actually removed risk, measured on realized returns rather than assumed from a beta.
`analyze_index_basket`	Value a basket of constituents against the index it replicates, attributing the spread name by name.
`analyze_index_rebalance`	The buying and selling an index change forces on passive money, sized as DAYS OF VOLUME first and currency second.
`analyze_roll`	What moving a position from one contract into the next actually costs, with the break-even rate it then has to out-earn.
`analyze_total_return_future`	Read a TRF quote as the financing spread it embeds, and compare it to what a swap charges.
`compare_delta_one_expressions`	Rank several ways of holding one exposure -- cash, ETF, future, forward, synthetic or swap -- on one annualized basis-point number.
`detect_basis_dislocation`	Whether a basis has STRUCTURALLY shifted rather than merely moved, by CUSUM and change-point detection.
`monitor_spread_stream`	Watch a spread on a LIVE feed, one stateful call at a time.
`optimize_replication_basket`	The smallest basket that tracks a benchmark, minimizing the variance of the DIFFERENCE rather than the portfolio's own variance.
`price_total_return_swap`	Mark a total return swap with the equity and financing legs separated.
`scan_basis_dislocations`	Rank many spot/futures pairs by how far each basis sits from its OWN history, not by how wide it is.
`size_futures_hedge`	The futures position that neutralizes a portfolio's beta, reported exact, rounded, and with the residual that rounding leaves.
`solve_forward_carry`	Recover whichever of financing, dividend or borrow a quoted forward implies, given the other two.

`microstructure` · Microstructure · 17 tools

What the market will charge you to trade, at two data fidelities.

`check_spread_proxy`	Measures the spread from ticks and compares it against the OHLCV estimate, so the proxy's error on THIS name is a number rather than an assumption.
`classify_trade_direction`	Sign a tick tape buyer- or seller-initiated and publish the signed series, which is what an event study, a CUSUM detector or a model consumes.
`detect_liquidity_events`	CUSUM change detection over tick-derived liquidity channels -- when the spread, depth, trade intensity or signed flow regime CHANGED, rather than what it is on average.
`estimate_corwin_schultz_spread`	The spread implied by the HIGH-LOW RANGE (Corwin-Schultz 2012).
`estimate_kyle_lambda`	Market DEPTH: the price impact of a unit of signed order flow, from a regression of price change on signed volume.
`estimate_roll_spread`	The effective spread implied by BID-ASK BOUNCE, from trade prices alone (Roll 1984).
`estimate_vpin`	Flow one-sidedness measured in VOLUME time rather than clock time (Easley, Lopez de Prado and O'Hara 2012).
`get_amihud_illiquidity`	How far the price moves per dollar traded (Amihud 2002) -- the most widely used liquidity proxy in the literature, because it needs nothing but daily bars.
`get_effective_spread_series`	What each trade ACTUALLY paid against the prevailing midpoint, per trade, published as a reference.
`get_implementation_shortfall`	What an execution ACTUALLY cost, decomposed after Perold.
`get_intraday_volume_profile`	How volume distributes across the trading day, and what that implies for a participation schedule.
`get_microstructure_metrics`	Quoted and effective spread MEASURED from trades and quotes, the effective spread split into realized (liquidity-provider) and impact (price-move) halves.
`get_order_book_metrics`	Depth-book statistics a top-of-book quote cannot give: the microprice, imbalance at the touch AND cumulatively, and how fast liquidity thins with distance.
`get_order_event_metrics`	Order-level statistics a depth book cannot produce: the queue AHEAD of an arriving order, order lifetimes, cancels per add and per trade, and event intensity by action.
`get_order_flow_imbalance`	Signed volume imbalance from bars, with its own predictive test attached rather than presented as a signal to be trusted.
`get_quoted_spread_series`	Spread and imbalance PER QUOTE rather than averaged into one number, published as a reference.
`get_trade_profile`	How volume distributes across trade sizes and times of day, from tick data.

`derivatives` · Derivatives · 13 tools

Price an option and understand what holding it does to you: the second-order greeks, multi-leg payoffs, the consistency of a quoted surface, what the market is pricing as a move, and what a delta hedge costs to run.

`aggregate_option_book`	Dollar delta, gamma per 1%, vega, theta per day and rho of options and underlying positions on several underlyings, per underlying and for the book, with a beta-weighted...
`analyze_option_strategy`	The payoff, breakevens and aggregate greeks of an arbitrary multi-leg position -- any combination of calls, puts and stock, rather than a fixed menu of named structures.
`analyze_vol_term_structure`	Contango or backwardation, and the FORWARD volatilities between expiries -- which is what a calendar spread actually prices.
`check_put_call_parity`	Whether a call and a put on the same strike are mutually consistent, and what the violation is worth.
`fit_volatility_smile`	Fit a quoted smile as a quadratic in LOG-MONEYNESS and check it for arbitrage.
`get_expected_move`	The move the option market is pricing over a horizon, with the standard misreading pre-empted.
`get_implied_forward`	The forward implied by carry, with financing, dividend and borrow broken out separately.
`get_implied_volatility`	The volatility that reproduces an observed option price.
`get_option_greeks`	The SECOND-ORDER greeks -- vanna, volga, charm and speed -- that explain why a delta-hedged book still loses money.
`get_option_pricing`	Price a European option and return its first-order greeks, under Black-Scholes, Black-76, Bachelier or a binomial tree.
`get_option_risk_scenarios`	A full REVALUATION grid over spot and volatility, not a delta-gamma approximation of one.

get_volatility_cone	Where today's implied vol sits inside this name's own history of REALIZED vol, horizon by horizon.
simulate_delta_hedge	What a delta-hedged short option actually earns when the vol you sold at is not the vol that shows up.

feature_lab · Feature Lab · 11 tools

Interrogate the FEATURES of a built dataset, before and independently of fitting anything: what each one measures and predicts, which are restatements of one another, whether one has drifted or only worked in one regime, whether its IC is larger than what this panel's noise produces, and what each is worth to a fitted model.

compare_feature_sets	Two feature sets measured on the same panel, with the cost of the difference attached: per-set IC, independent-signal count and condition number, and what is unique to each...
get_feature_drift	Whether a feature is still the same measurement, and still predicts, either side of a date.
get_feature_ic_decay	How one feature's IC behaves when the feature is displaced in time.
get_feature_redundancy	Which features are restatements of one another, and which one to keep.
get_feature_regime_stability	The feature's IC inside each of several CONTIGUOUS time blocks, never shuffled.
profile_feature	Profile ONE feature of a built dataset: coverage, turnover, autocorrelation, cross-sectional IC and ICIR, quantile spread and monotonicity.
run_feature_ablation	Refit the model without each feature in turn and report what each one was worth.
run_feature_permutation_test	How often a null drawn from THIS panel produces an IC as far from zero as the observed one, for one feature.
screen_feature_significance	run_feature_permutation_test for EVERY candidate feature at once, and the noise level the widest null sets.
screen_feature_stability	Every feature's drift and every feature's regime dependence in one pass: PSI and KS across a split, the IC on each side, the IC inside each contiguous block and sign...
select_features	Choose a feature set from a built dataset: keep one feature per redundancy cluster, apply an optional IC floor, and drop what does not pass a permutation test against the...

allocation · Capital Allocation · 10 tools

How much capital each strategy, sleeve or model deserves, from their return streams rather than from tickers: the streams on one calendar, an allocation with its weight bands, a walk-forward test of the rule that chose it against an equal-weight benchmark, the Kelly fraction and how uncertain it is, a volatility-target or CPPI overlay, the capacity curve under impact, Brinson attribution and a...

allocate_across_sleeves	Splits capital across strategies, sleeves or models by their return streams: equal, inverse volatility, risk parity, HRP, min variance, max Sharpe or shrunk Sharpe.
apply_risk_overlay	Runs a volatility-target (lagged EWMA), CPPI or TIPP overlay over one return stream: realized against target volatility, drawdown before and after, floor breaches and...
attribute_returns_brinson	Brinson-Fachler allocation, selection and interaction per segment and period, linked by Carino or Menchero so the effects sum to the cumulative active return over a benchmark.
build_model_sleeves	Evaluates two to twenty registered models as portfolios and publishes their out-of-sample return streams on one calendar as a returns_panel, with the registry's trial count.
combine_return_streams	Puts several return streams on the dates they all share and publishes the aligned panel as one returns_panel reference, with what each stream lost to the common calendar.
evaluate_allocation_rule	Walks an allocation rule forward over sleeve return streams -- fit on a trailing window, drift, rebalance, pay the cost -- and scores it out of sample against a benchmark...
get_kelly_report	The Kelly leverage of one return stream (mu / sigma^2) or several held together (Sigma^-1 mu), its standard error and the chance the estimate is over twice the truth.
get_strategy_capacity_curve	Net return and Sharpe ratio against AUM for a strategy's gross return stream and weight panel under spread plus square-root impact, with its break-even and half-Sharpe AUM.
measure_book_returns	Time-weighted, linked Modified Dietz and money-weighted returns of a portfolio with deposits and withdrawals (GIPS 2020), from dated valuations and dated external flows.
select_allocation_rule	Chooses among 2-12 allocation rules as it goes, at each rebalance by their own out-of-sample record before that date, and tests that choosing against equal weight out of...

`finance` · Finance · 9 tools

What holding a book costs and what a stream of cash is worth: financing, borrow and cash drag on positions, the Reg T and FINRA margin against them and how long a stress path leaves before the first call, currency exposure, carry and the FX forward that hedges it, and NPV, IRR, duration, a cost of capital and a discounted valuation.

`analyze_book_carry`	Annual carry of each position and of the set in the base currency (Koijen, Moskowitz, Pedersen & Vrugt 2018).
`analyze_cash_flows`	NPV, IRR, MIRR, Macaulay and modified duration, convexity and DV01 of periodic or dated cash flows, listing every real IRR and giving the IRR only when it is unique.
`analyze_margin_runway`	Walks a stress path you supply: the first maintenance call and its amount, the day forced deleveraging starts if calls go unmet, and the least cash today that avoids every...
`compute_margin_requirement`	Reg T initial and FINRA 4210 maintenance margin per position and for the account, each line naming its rule: stocks, leveraged ETFs and uncovered equity options.
`estimate_cost_of_capital`	CAPM cost of equity, after-tax cost of debt and WACC on market values, with Hamada's unlevered beta and the beta and WACC relevered at a target debt-to-equity.
`estimate_financing_cost`	What a set of positions costs to finance over a horizon: debit and credit interest, short borrow, the rebate, the net cost, and its rate on gross exposure and on equity.
`get_currency_exposure`	Currency exposure of a set of positions in the base currency, net and gross per currency after FX forward or spot hedges, with each currency's hedge ratio.
`price_fx_forward_hedge`	Covered-interest-parity FX forward of a pair, each leg on its own day count plus any cross-currency basis, its points in pips, and the annual pickup or cost of the hedge.
`value_cash_flows`	DCF value of forecast free cash flows with a Gordon or exit-multiple terminal value, the terminal value's share of it, and a value grid by discount rate and growth or...

Standard Quant Tools

The agent chooses. The library computes.

Every call can be replayed.

TYPED

One Pydantic model in, one out, for every one of the 266 tools. Unknown arguments are refused, and every refusal names the fix.

SCOPED

Twelve runtimes, each its own dispatch table. A client is served one, and a tool it was not given is unroutable.

MEASURED

Planted answers, a second oracle per kernel, fuzzing, mutation testing, and the results that did not help kept beside the ones that did.

Start here

pip install . · sqt-mcp --runtime research

The guides, the generated tool index and this briefing's sources are in the repository.

Karan Vora, Maintainer · github.com/karanvora2599/Standard-Tools

Standard Quant Tools is open source under Apache 2.0. Its agent surface is a typed layer over the same library a person calls from Python.

A software library, not investment advice. Every figure in this briefing was measured from the code on 27 October 2026, at commit 09727c7.



Karan Vora, Maintainer · Apache 2.0

github.com/karanvora2599/Standard-Tools